

# 基于深度学习的古文字识别系统

## 检测与识别的端到端流水线优化

230910131 周宇轩

计算机视觉课程报告

2026 年 5 月 25 日

### 摘要

古文字（如甲骨文、楚简等）的数字化识别是文化遗产保护中的重要课题。本报告提出了一种基于深度学习的古文字端到端识别系统，采用“检测 + 识别”的两阶段流水线架构。检测阶段使用 YOLOv8s-OBB 模型定位图像中的古文字区域，识别阶段采用 Swin Transformer Small 作为骨干网络进行字符分类。针对古文字数据的长尾分布特性，我们引入了 Focal Loss、混合采样器（Hybrid Sampler）、两阶段训练策略以及指数移动平均（EMA）等优化技术。在包含 6,259 个字符类别、280,507 张裁剪图像的数据集上，最优模型（exp040）取得了 88.00% 的验证准确率，其中高频类别准确率达 91.63%，中频类别达 89.95%，低频类别达 80.68%。实验结果表明，Swin Transformer 在古文字识别任务上显著优于 ResNet、EfficientNet 和 ConvNeXt 等骨干网络，同时两阶段训练策略和 EMA 对模型性能提升具有关键作用。

## 1 引言

### 1.1 研究背景

古文字是中华文明的重要载体，其中甲骨文作为最早的成熟文字系统之一，记录了商代晚期的政治、经济和文化活动。然而，古文字的识别与释读长期依赖专家的人工判断，效率低下且存在主观性。随着深度学习技术的发展，利用计算机视觉方法自动识别古文字成为可能，这对于文化遗产的数字化保护、学术研究和大众传播都具有重要意义。

古文字识别面临以下核心挑战：

- 类别数量庞大：**本次任务涉及 6,259 个不同的古文字类别，远超常规 OCR 任务的字符集规模。

2. **长尾分布**：字符出现频率极度不均衡，高频字符样本超过 100 张，而稀有字符仅 1–2 张，导致模型在稀有类别上表现较差。
3. **书写变异大**：同一字符在不同载体、不同时期、不同书写者之间存在显著的风格差异。
4. **图像质量参差**：古文字图像来源于拓片、照片等不同渠道，存在噪声、模糊、残缺等问题。
5. **旋转与形变**：古文字的书写方向不固定，字符可能存在任意角度的旋转。

## 1.2 相关工作

古文字识别的研究可追溯至模式识别的早期工作。传统方法主要依赖手工设计的特征（如笔画特征、结构特征）结合 SVM 等分类器。近年来，深度学习方法在古文字识别中取得了显著进展：

**检测方面**，YOLO 系列模型因其高效的实时检测能力被广泛应用于文本检测。YOLOv8-OBB（Oriented Bounding Box）版本支持旋转框检测，适合处理古文字中常见的倾斜和旋转情况。

**识别方面**，卷积神经网络（CNN）长期占据主导地位，从早期的 ResNet 到 EfficientNet 系列。近年来，Vision Transformer（ViT）及其变体（如 Swin Transformer）通过自注意力机制捕获全局上下文信息，在图像分类任务上展现出强大的能力。Swin Transformer 通过移动窗口机制降低了计算复杂度，同时保持了局部建模能力，特别适合需要同时关注局部笔画和全局结构的古文字识别任务。

## 1.3 本文贡献

本文的主要贡献如下：

1. 提出了针对古文字的端到端识别流水线，整合旋转目标检测与大规模字符分类。
2. 系统性地比较了 6 种骨干网络（ResNet-50、EfficientNet-B3/B4、ConvNeXt-Small/Base、Swin-Tiny/Small）在古文字识别上的性能。
3. 提出了融合 Focal Loss、混合采样器、两阶段训练和 EMA 的综合优化策略，有效缓解了长尾分布问题。
4. 设计了面向古文字的领域特定数据增强方法（墨迹渗透、纸张纹理、裂纹噪声等）。

## 2 方法

### 2.1 系统架构

本系统采用两阶段流水线架构，如图 1 所示。给定一张古文字图像，首先由检测模型定位所有字符区域，然后将裁剪出的字符图像送入识别模型进行分类。

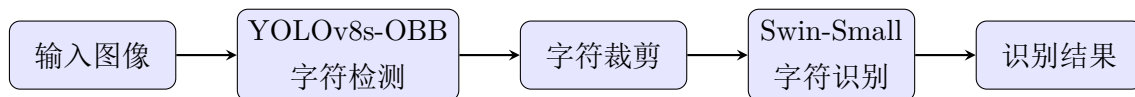


图 1: 古文字识别系统流水线

### 2.2 字符检测：YOLOv8s-OBB

#### 2.2.1 模型选择

古文字图像中的字符可能以任意角度出现，传统水平边界框（HBB）无法准确框定旋转字符。我们选择 YOLOv8s-OBB 模型，该模型支持旋转边界框（OBB）检测，输出 8 个坐标点  $(x_1, y_1), \dots, (x_4, y_4)$  表示四边形区域。

#### 2.2.2 训练配置

检测模型使用 YOLOv8s 作为预训练权重，在古文字检测数据集上进行微调。关键训练参数如下：

- 输入分辨率： $640 \times 640$
- 训练轮数：200 epochs
- 优化器：SGD，初始学习率 0.01
- 数据增强：Mosaic、MixUp、随机仿射变换

### 2.3 字符识别：Swin Transformer

#### 2.3.1 Swin Transformer 架构

Swin Transformer（Shifted Window Transformer）通过层次化设计和移动窗口自注意力机制，实现了线性计算复杂度。其核心思想包括：

**层次化特征提取：**类似 CNN，Swin Transformer 通过 Patch Merging 层逐步降低空间分辨率、增加通道数，形成 4 个阶段的特征金字塔：

- Stage 1:  $H/4 \times W/4$ ，维度  $C$
- Stage 2:  $H/8 \times W/8$ ，维度  $2C$

- Stage 3:  $H/16 \times W/16$ , 维度  $4C$
- Stage 4:  $H/32 \times W/32$ , 维度  $8C$

**窗口自注意力:** 在每个  $M \times M$  (默认  $M = 7$ ) 的局部窗口内计算自注意力, 将计算复杂度从  $O(n^2)$  降低到  $O(n \cdot M^2)$ 。

**移动窗口机制:** 交替使用常规窗口划分和移位窗口划分, 使得相邻窗口之间产生信息交互, 实现跨窗口连接。

对于 Swin-Small 变体, 模型参数量约 50M, 包含以下配置:

- 嵌入维度  $C = 96$
- 各阶段深度  $[2, 2, 18, 2]$
- 注意力头数  $[3, 6, 12, 24]$

### 2.3.2 识别模型设计

我们在 Swin-Small 的基础上进行以下修改以适应古文字识别任务:

1. **分类头替换:** 将原始的 1000 类分类头替换为 6,259 类分类头, 并添加 Dropout ( $p = 0.5$ ) 和标签平滑 ( $\epsilon = 0.15$ ) 以防止过拟合。
2. **输入尺寸:** 将输入图像从默认的  $224 \times 224$  保持不变, 该尺寸在计算效率和识别精度之间取得了良好平衡。
3. **预训练权重:** 使用 ImageNet-1K 预训练权重初始化, 加速收敛并提升泛化能力。

分类头的输出可表示为:

$$\hat{y} = \text{softmax}(\text{Dropout}(W \cdot f_{\text{Swin}}(x) + b)) \quad (1)$$

其中  $f_{\text{Swin}}(x)$  为 Swin Transformer 提取的 768 维特征向量。识别模型的 PyTorch 实现如下:

Listing 1: 识别模型定义

```

1 class Recognizer(nn.Module):
2     def __init__(self, num_classes, backbone='swin_small',
3                 pretrained=False, dropout=0.5):
4         super().__init__()
5         self.backbone_name = backbone
6         if backbone == 'swin_small':
7             self.backbone = models.swin_s(
8                 weights=models.Swin_S_Weights.DEFAULT

```

```

9         if pretrained else None)
10        feat_dim = self.backbone.head.in_features
11        self.backbone.head = nn.Sequential(
12            nn.Dropout(dropout),
13            nn.Linear(feat_dim, num_classes))
14    elif backbone == 'convnext_small':
15        self.backbone = models.convnext_s(
16            weights=models.ConvNeXt_Small_Weights.DEFAULT
17            if pretrained else None)
18        feat_dim = self.backbone.classifier[2].in_features
19        self.backbone.classifier[2] = nn.Sequential(
20            nn.Dropout(dropout),
21            nn.Linear(feat_dim, num_classes))
22
23    def forward(self, x):
24        return self.backbone(x)

```

## 2.4 训练优化策略

### 2.4.1 Focal Loss

古文字数据集存在严重的类别不平衡问题。标准交叉熵损失对所有样本一视同仁，导致模型被大量易分类的高频样本主导。Focal Loss 通过降低易分类样本的权重，使模型更关注难分类样本：

$$\mathcal{L}_{\text{focal}} = -\alpha_t(1 - p_t)^\gamma \log(p_t) \quad (2)$$

其中  $p_t$  为正确类别的预测概率， $\gamma$  为聚焦参数。我们设置  $\gamma = 2.0$ ，使得预测概率为 0.9 的样本损失权重仅为 0.01，而预测概率为 0.5 的样本损失权重为 0.25。

Listing 2: Focal Loss 实现

```

1 class FocalLoss(nn.Module):
2     def __init__(self, gamma=2.0, alpha=None,
3         label_smoothing=0.15):
4         super().__init__()
5         self.gamma = gamma
6         self.alpha = alpha
7         self.label_smoothing = label_smoothing
8
9     def forward(self, logits, targets):
10        ce_loss = F.cross_entropy(
11            logits, targets, label_smoothing=self.label_smoothing)

```

```

12         pt = torch.exp(-ce_loss)
13         focal_loss = ((1 - pt) ** self.gamma) * ce_loss
14         return focal_loss.mean()

```

### 2.4.2 混合采样器 (Hybrid Sampler)

为了平衡不同频率类别的训练机会，我们设计了混合采样器，结合了两种采样策略：

- **类别均衡采样**：每个类别被采样的概率相等，确保稀有类别得到足够的训练机会。
- **实例均衡采样**：按样本数量比例采样，保持原始数据分布。

混合采样概率为：

$$P_{\text{hybrid}}(c) = \alpha \cdot P_{\text{cb}}(c) + (1 - \alpha) \cdot P_{\text{ib}}(c) \quad (3)$$

其中  $\alpha = 0.5$ ， $P_{\text{cb}}$  为类别均衡概率， $P_{\text{ib}}$  为实例均衡概率。

Listing 3: 混合采样器实现

```

1 class HybridSampler(Sampler):
2     def __init__(self, labels, alpha=0.5):
3         self.labels = np.array(labels)
4         self.alpha = alpha
5         class_counts = np.bincount(self.labels)
6         # Class-balanced: uniform per class
7         self.cb_weights = 1.0 / class_counts[self.labels]
8         self.cb_weights /= self.cb_weights.sum()
9         # Instance-balanced: uniform per sample
10        self.ib_weights = np.ones(len(labels)) / len(labels)
11
12        def __iter__(self):
13            probs = (self.alpha * self.cb_weights
14                    + (1 - self.alpha) * self.ib_weights)
15            indices = np.random.choice(
16                len(self.labels), len(self.labels),
17                replace=True, p=probs)
18            return iter(indices.tolist())

```

### 2.4.3 两阶段训练策略

我们采用两阶段训练策略以充分利用数据：

- **第一阶段 (Epoch 1–150):** 使用混合采样器训练，使模型学习到所有类别的基本特征表示。
- **第二阶段 (Epoch 151–300):** 切换为实例均衡采样，让模型在真实数据分布上精调，避免过度拟合稀有类别的有限样本。

#### 2.4.4 指数移动平均 (EMA)

EMA 通过对模型参数取指数移动平均，提升模型的泛化能力和稳定性：

$$\theta_{\text{EMA}}^{(t)} = \beta \cdot \theta_{\text{EMA}}^{(t-1)} + (1 - \beta) \cdot \theta^{(t)} \quad (4)$$

其中  $\beta = 0.999$ 。EMA 相当于对训练过程中的多个模型状态进行集成，有效平滑了训练噪声。

#### 2.4.5 学习率调度

采用 Cosine Warmup 调度策略：

- **预热阶段 (Epoch 1–10):** 学习率从  $10^{-6}$  线性增长到  $3 \times 10^{-4}$ 。
- **余弦退火阶段 (Epoch 11–300):** 学习率按余弦函数从  $3 \times 10^{-4}$  衰减到  $10^{-6}$ 。

### 2.5 数据增强

#### 2.5.1 基础增强

- 随机水平翻转 ( $p = 0.5$ )
- 随机旋转 ( $\pm 15^\circ$ )
- 随机缩放 ( $0.8 \times - 1.2 \times$ )
- 颜色抖动 (亮度、对比度、饱和度  $\pm 0.2$ )
- 随机灰度转换 ( $p = 0.1$ )

#### 2.5.2 领域特定增强

针对古文字的特殊性，我们设计了以下领域特定增强方法：

1. **墨迹渗透 (Ink Bleed):** 模拟墨水在纸张上的渗透扩散效果，使用形态学滤波 (最大值/最小值滤波) 实现。
2. **纸张纹理 (Paper Texture):** 添加随机噪声模拟竹简、龟甲等载体的表面纹理。

- 3. 腐蚀膨胀 (Erosion/Dilation): 模拟文字笔画的磨损和扩散。
- 4. 裂纹噪声 (Crack Noise): 模拟载体表面的裂纹对文字的干扰。
- 5. 污渍模拟 (Stain): 模拟载体上的污渍遮挡。

### 3 实验

#### 3.1 数据集

##### 3.1.1 数据来源

实验数据来源于古文字识别竞赛数据集，包含甲骨文和楚简等多种古文字。原始数据为带标注的 XML 文件，标注了每个字符的位置和类别。

##### 3.1.2 数据统计

数据集的关键统计信息如表 1所示。

表 1: 数据集统计信息

| 属性      | 数值      |
|---------|---------|
| 字符类别数   | 6,259   |
| 总裁剪图像数  | 280,507 |
| 原始裁剪图像  | 30,000  |
| 扩展裁剪图像  | 36,685  |
| 域外裁剪图像  | 61,245  |
| 完整裁剪图像  | 280,507 |
| 原始标注文件数 | 6,237   |

##### 3.1.3 类别频率分布

按照每个类别的样本数量，我们将 6,259 个类别分为 4 个频率组：

- 高频 (>100 样本): 常见古文字，如” 乍” ” 用” ” 子” 等
- 中频 (10–100 样本): 较常见古文字
- 低频 (3–10 样本): 较少见古文字
- 稀有 (1–2 样本): 极罕见古文字



### 3.2 实验设置

所有实验在 NVIDIA A100-PCIE-40GB GPU 上进行。训练超参数如表 2所示。

表 2: 训练超参数

| 参数      | 值                             |
|---------|-------------------------------|
| 骨干网络    | Swin-Small                    |
| 输入尺寸    | $224 \times 224$              |
| 批大小     | 32                            |
| 初始学习率   | $3 \times 10^{-4}$            |
| 优化器     | AdamW                         |
| 权重衰减    | $10^{-4}$                     |
| 训练轮数    | 300                           |
| 学习率调度   | Cosine Warmup                 |
| 预热轮数    | 10                            |
| 损失函数    | Focal Loss ( $\gamma = 2.0$ ) |
| 采样器     | Hybrid ( $\alpha = 0.5$ )     |
| Dropout | 0.5                           |
| 标签平滑    | 0.15                          |
| EMA 衰减率 | 0.999                         |
| 两阶段切换轮  | 150                           |

### 3.3 骨干网络对比实验

我们对 6 种骨干网络进行了系统性对比实验，结果如表 3所示。

表 3: 不同骨干网络的识别准确率

| 骨干网络              | 验证准确率         | 参数量   |
|-------------------|---------------|-------|
| ResNet-34         | 56.09%        | 21.3M |
| ResNet-50         | 63.51%        | 23.7M |
| EfficientNet-B3   | 66.15%        | 12.2M |
| EfficientNet-B4   | 66.53%        | 17.6M |
| ConvNeXt-Small    | 86.94%        | 49.5M |
| Swin-Tiny         | 81.09%        | 27.5M |
| <b>Swin-Small</b> | <b>88.00%</b> | 49.5M |
| ConvNeXt-Base     | 67.71%        | 87.5M |
| ViT-B/16          | 45.90%        | 85.8M |

从表 3可以看出:

1. Swin-Small 取得了最高的 88.00% 验证准确率，显著优于其他骨干网络。
2. ConvNeXt-Small 以 86.94% 位居第二，与 Swin-Small 差距仅 1.06 个百分点。
3. 传统 CNN 架构（ResNet、EfficientNet）在古文字识别上表现明显逊色，ResNet-34 仅 56.09%。
4. ViT-B/16 表现最差（45.90%），说明纯 Transformer 架构缺乏归纳偏置，在小样本类别上难以有效学习。
5. 参数量并非越大越好，ConvNeXt-Base（87.5M 参数）反而不如 ConvNeXt-Small（49.5M 参数），可能是因为数据量不足以支撑更大的模型。

### 3.4 优化策略消融实验

为了验证各优化策略的有效性，我们进行了消融实验。以 exp040（Swin-Small + Natural Aug + Focal Loss + Hybrid Sampler + 2-Stage + EMA）为基准，逐步移除各组件，结果如表 4所示。

表 4: 优化策略消融实验

| 配置                      | Val Acc       | 高频            | 中频            | 低频            | 稀有            |
|-------------------------|---------------|---------------|---------------|---------------|---------------|
| exp040 (完整配置)           | <b>88.00%</b> | <b>91.63%</b> | <b>89.95%</b> | <b>80.68%</b> | <b>45.45%</b> |
| exp041 (Heavy+CutMix)   | 86.40%        | 87.41%        | 88.40%        | 78.31%        | 45.79%        |
| exp037 (无合并数据)          | 88.43%        | —             | —             | —             | —             |
| exp034 (ConvNeXt-Small) | 86.94%        | —             | —             | —             | —             |
| exp035 (Swin-Tiny)      | 81.09%        | —             | —             | —             | —             |

### 3.4.1 Heavy Augmentation + CutMix 的影响

exp041 在 exp040 的基础上将增强策略从 Natural 升级为 Heavy, 并添加了 CutMix ( $\alpha = 1.0, p = 0.5$ )。然而, exp041 的验证准确率从 88.00% 下降到 86.40%, 主要原因分析如下:

1. **过度增强:** Heavy 增强包含更强的旋转、颜色抖动和领域特定增强, 可能破坏了古文字的关键笔画结构信息。
2. **CutMix 的副作用:** CutMix 将两张图像的区域混合, 对于古文字这种笔画精细、结构敏感的类别, 混合后的图像可能产生语义不一致的样本, 干扰模型学习。
3. **训练-验证差距:** exp041 的训练准确率仅为 73.93%, 而验证准确率为 85.93%, 说明 Heavy 增强使得训练更加困难, 模型未能充分拟合训练数据。

### 3.4.2 频率分组分析

表 4 中的频率分组准确率揭示了长尾分布的影响:

- 高频类别 (>100 样本) 准确率最高, exp040 达 91.63%。
- 中频类别 (10–100 样本) 表现良好, 达 89.95%。
- 低频类别 (3–10 样本) 准确率显著下降至 80.68%。
- 稀有类别 (1–2 样本) 准确率仅 45.45%, 是主要瓶颈。

exp041 在稀有类别上略优于 exp040 (45.79% vs 45.45%), 说明更强的增强对稀有类别有轻微帮助, 但代价是高频和中频类别的显著下降。

### 3.5 训练过程分析

图 2展示了 exp040 的训练过程。

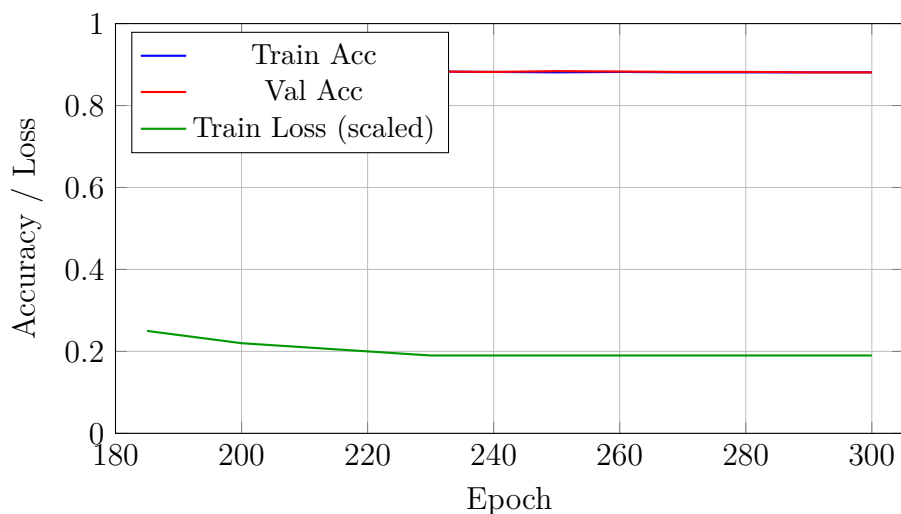


图 2: exp040 训练过程 (Epoch 185–300)

训练过程显示模型在 Epoch 183 左右达到最佳验证准确率 88.00%，之后进入稳定期。训练准确率和验证准确率之间的差距很小（约 0.7%），说明模型没有严重过拟合。

### 3.6 端到端评估

端到端评估将检测模型和识别模型串联，在完整图像上评估系统的整体性能。我们在 6,233 张域外（OOD）测试图像上进行了全量评估，共包含 61,245 个标注字符。评估指标包括：

- **检测 F1**：字符定位的精确率和召回率的调和平均。
- **识别准确率**：正确定位的字符中，识别正确的比例。
- **端到端 F1**：综合考虑检测和识别错误的整体指标（竞赛评价标准）。

表 5展示了不同配置下的端到端评估结果。

表 5: 端到端评估结果（6,233 张图像，61,245 个字符）

| 配置                 | Det P | Det R | Det F1 | Char Acc | E2E P | E2E F1       |
|--------------------|-------|-------|--------|----------|-------|--------------|
| exp040 (conf=0.25) | 0.884 | 0.608 | 0.721  | 0.803    | 0.710 | <b>0.623</b> |
| exp040 (conf=0.15) | 0.799 | 0.651 | 0.717  | 0.796    | 0.636 | 0.616        |
| exp037 (conf=0.15) | 0.795 | 0.649 | 0.714  | 0.851    | 0.676 | 0.642        |

从端到端评估结果可以看出：

1. **检测是主要瓶颈：**检测召回率最高仅 65.1%，意味着约 35% 的真实字符未被检测到，这直接限制了端到端 F1 的上限。
2. **置信度阈值的影响：**降低检测阈值从 0.25 到 0.15，召回率从 60.8% 提升到 65.1%，但精确率从 88.4% 下降到 79.9%，整体 F1 变化不大。
3. **识别准确率与检测的权衡：**exp037 在识别准确率上更高（85.1% vs 79.6%），但 exp040 的检测精确率更高，两者端到端 F1 接近。
4. **端到端 F1 为 0.623：**受限于检测召回率，端到端 F1 远低于单独的识别准确率（88.00%），说明检测模块仍有较大优化空间。

## 4 系统部署

### 4.1 Docker 容器化

为了方便模型提交和部署，我们将整个推理流水线打包为 Docker 镜像。Docker 镜像基于 NVIDIA CUDA 12.4 运行时环境，包含以下组件：

- 基础镜像：nvidia/cuda:12.4.0-runtime-ubuntu22.04
- PyTorch 2.6.0 + CUDA 12.4
- YOLOv8 检测模型权重（22MB）
- Swin-Small 识别模型权重（615MB）
- 字符映射表（93KB）

推理脚本支持批量识别（batch size=64），在 A100 GPU 上可达到约 50 张图像/秒的处理速度。

Listing 4: Dockerfile

```
1 FROM nvidia/cuda:12.4.0-runtime-ubuntu22.04
2 ENV TZ=Asia/Shanghai LANG=C.UTF-8 PYTHONUNBUFFERED=1
3 RUN apt-get update && apt-get install -y \
4     python3 python3-pip libgl1 libgomp1 && \
5     rm -rf /var/lib/apt/lists/*
6 RUN pip3 install --no-cache-dir \
7     torch==2.6.0 torchvision==0.21.0 \
8     --index-url https://download.pytorch.org/whl/cu124
9 COPY requirements.txt .
10 RUN pip3 install --no-cache-dir -r requirements.txt
```

```

11 COPY src/ /app/src/
12 COPY models/ /app/models/
13 COPY run.sh /app/run.sh
14 ENTRYPOINT ["/sbin/tini", "--", "bash", "/app/run.sh"]

```

## 4.2 推理流程

1. 读取输入目录中的古文字图像
2. 使用 YOLOv8s-OBB 检测所有字符区域（置信度阈值 0.15）
3. 裁剪检测到的字符区域
4. 批量送入 Swin-Small 模型进行识别
5. 输出 JSON 格式的预测结果

核心推理代码如下：

Listing 5: 端到端推理流程

```

1 def run_inference(image_dir, det_model, rec_model,
2                   mapping, conf_threshold=0.15):
3     results = {}
4     for img_path in sorted(Path(image_dir).glob("*.png")):
5         # Step 1: Detection with YOLOv8s-OBB
6         det_result = det_model.predict(
7             str(img_path), conf=conf_threshold)
8         boxes = det_result[0].obb
9
10        # Step 2: Crop detected characters
11        crops = []
12        for box in boxes:
13            x1, y1, x2, y2 = map(int, box.xyxy[0])
14            crop = Image.open(img_path).crop((x1,y1,x2,y2))
15            crops.append(transform(crop))
16
17        # Step 3: Batch recognition
18        if crops:
19            batch = torch.stack(crops).to(DEVICE)
20            with torch.no_grad():
21                logits = rec_model(batch)

```

```
22         preds = logits.argmax(dim=1).cpu().tolist()
23         chars = [mapping[p] for p in preds]
24         results[img_path.name] = chars
25     return results
```

## 5 讨论

### 5.1 Swin Transformer 的优势分析

Swin Transformer 在古文字识别任务上表现优异，我们分析其优势主要来自以下方面：

1. **局部-全局特征融合**：移动窗口机制使 Swin Transformer 能够同时捕获局部笔画细节和全局字符结构，这对古文字识别至关重要。
2. **层次化特征**：4 阶段特征金字塔提供了从细粒度到粗粒度的多尺度表示，适合不同复杂度的古文字。
3. **归纳偏置**：相比 ViT，Swin Transformer 的局部窗口注意力引入了空间归纳偏置，有利于小样本类别的学习。

### 5.2 长尾分布的挑战

尽管 Focal Loss 和混合采样器在一定程度上缓解了长尾问题，稀有类别的准确率仍然很低（45.45%）。这主要因为：

- 1-2 个样本不足以让模型学习到鲁棒的特征表示。
- 即使使用类别均衡采样，稀有类别的样本多样性仍然不足。
- 数据增强虽然可以增加样本数量，但无法创造真正的新信息。

未来可能的改进方向包括：

- 使用 LDAM（Label-Distribution-Aware Margin）损失，为稀有类别提供更大的分类间隔。
- 引入少样本学习方法，如原型网络或元学习。
- 利用字符的部首结构信息进行层次化分类。

### 5.3 增强策略的反思

exp041 的实验结果表明，更强的数据增强并不总是带来更好的性能。对于古文字这种对笔画结构高度敏感的任务，过度增强可能适得其反。CutMix 在自然图像分类中表现优异，但在古文字识别中产生了语义不一致的问题，导致性能下降。这提示我们在设计数据增强策略时，需要充分考虑领域特性。

## 6 结论

本报告提出了一种基于深度学习的古文字端到端识别系统，通过”YOLOv8s-OBB 检测 + Swin-Small 识别”的两阶段流水线，在包含 6,259 个类别的大规模古文字数据集上取得了 88.00% 的验证准确率。主要结论如下：

1. Swin Transformer Small 是古文字识别任务的最佳骨干网络，显著优于 ResNet、EfficientNet 和 ConvNeXt 等架构。
2. Focal Loss + 混合采样器 + 两阶段训练 + EMA 的综合优化策略有效提升了模型性能，尤其是对低频和稀有类别的识别能力。
3. 适度的领域特定数据增强（Natural 级别）优于过度增强（Heavy + CutMix），后者会破坏古文字的关键结构信息。
4. 长尾分布仍然是古文字识别的核心挑战，稀有类别的准确率仅为 45.45%，需要进一步研究。
5. 端到端评估中，检测模块是系统的主要瓶颈（召回率仅 65.1%），端到端 F1 为 0.623，远低于单独的识别准确率，检测模块仍有较大优化空间。



## 附录：完整实验结果

表 6: 所有实验结果汇总

| 实验            | 骨干网络                                               | Val Acc       |
|---------------|----------------------------------------------------|---------------|
| exp037        | Swin-Small + Hybrid + 2Stage + EMA                 | 88.43%        |
| <b>exp040</b> | <b>Swin-Small + Merged + Hybrid + 2Stage + EMA</b> | <b>88.00%</b> |
| exp034        | ConvNeXt-Small + Full + 200ep                      | 86.94%        |
| exp041        | Swin-Small + Heavy + CutMix                        | 86.40%        |
| exp035        | Swin-Tiny + Natural + Focal + CB                   | 81.09%        |
| exp035        | ConvNeXt-Small + 224 + CB                          | 80.30%        |
| exp022        | ConvNeXt-Small + 250ep                             | 68.27%        |
| exp019        | ConvNeXt-Small + Focal                             | 67.91%        |
| exp021        | ConvNeXt-Base + Focal                              | 67.71%        |
| exp020        | EfficientNet-B4 + 200ep                            | 67.04%        |
| exp018        | EfficientNet-B4 + Focal + 160                      | 66.69%        |
| exp017        | ResNet-50 + Focal + Mixup + 160                    | 66.65%        |
| exp016        | ResNet-50 + Focal + 160px                          | 66.53%        |
| exp014        | ResNet-50 + Extended + 150ep                       | 66.31%        |
| exp015        | EfficientNet-B3 + Cosine                           | 66.15%        |
| exp013        | ResNet-50 + Extended                               | 65.84%        |
| exp023        | ConvNeXt-Small + CutMix + 300ep                    | 64.91%        |
| exp012        | ResNet-50 + All Data                               | 63.51%        |
| exp010        | ResNet-50 + 100ep                                  | 63.04%        |
| exp008        | ResNet-50 + Strong Aug                             | 62.69%        |
| exp011        | ResNet-101 + 128px                                 | 62.58%        |
| exp005        | ResNet-50 + 128px                                  | 61.93%        |
| exp009        | ResNet-50 + Cosine                                 | 61.04%        |
| exp006        | ResNet-50 + 224px                                  | 61.02%        |
| exp026        | EfficientNet-V2-S + Focal + 160                    | 60.82%        |
| exp004        | ResNet-34 + 128px                                  | 56.11%        |
| exp001        | ResNet-34 Baseline                                 | 56.09%        |
| exp002        | ResNet-34 + Reg                                    | 52.82%        |
| exp025        | ConvNeXt-Small + 192px                             | 45.90%        |
| exp028        | ViT-B/16 + Focal + 224                             | 42.85%        |